

Software Reengineering

Software Changes: Maintenance, Evolution and Re-engineering

Week-3

Software Change

Understand three different strategies for changing software systems namely

1. Software Maintenance
2. Software Evolution
3. Software Re-engineering

Software Maintenance and Evolution

Maintenance

- The concept of **software maintenance** means preventing software from failing to deliver the intended functionalities by means of bug fixing.
- All support activities carried out after delivery of software are put under the category of maintenance.

Evolution

- The concept of **software evolution** means a continual change from a lesser, simpler, or worse state to a higher or better state.
- All activities carried out to effect changes in requirements are put under the category of evolution.

Software Maintenance

- Maintenance of software systems primarily means fixing bugs but preserving their functionalities.
- Maintenance tasks are very much planned.
- For example, bug fixing must be done and it is a planned activity.
- In addition to the planned activities, unplanned activities are also required.
- For example, a new usage of the system may emerge.
- Generally, maintenance does not involve making major changes to the architecture of the system.
- In other words, maintenance means keeping an installed system running with no change to its design.

Software Evolution

- Evolution of software systems means creating new but related designs from existing ones.
- The objectives include supporting new functionalities, making the system perform better, and making the system run on a different operating system.
- Basically, as time passes, the stakeholders develop more knowledge about the system.
- Therefore, the system evolves in several ways. As time passes, not only new usages emerge, but also the users become more knowledgeable.
- Over time what evolves is not the software but our knowledge about a particular type of software.

Software Re-engineering

- Evolution: The process of conducting continuous software reengineering”.
- Reengineering implies a single cycle of taking an existing system and generating from it a new system, whereas evolution can go forever.
- In other words, to a large extent, software evolution be seen as repeated software reengineering.
- Reengineering is done to transform an existing “lesser or simpler” system into a new “better” system.

Objectives

Software systems are reengineered by keeping one or more of the following four general objectives in mind:

1. Improving maintainability
2. Migrating to a new technology
3. Improving quality
4. Preparing for functional enhancement.

Re-engineering Risks

- Reengineering is done to convert an existing “bad” system into a “good” system.
- There are risks involved in this transformation, and the primary risks are:
 1. the target system may not have the same functionality as the existing system;
 1. the target system may be of lower quality than the original system;
 2. the benefits are not realized in the required time frame

Reengineering Concepts

Abstraction and **Refinement** are key concepts used in software development, and both the concepts are equally useful in reengineering.

It may be recalled that abstraction enables software maintenance personnel to **reduce the complexity of understanding a system** by:

- (i) focusing on the more significant information about the system; and
- (ii) Hiding the irrelevant details at the moment.

On the other hand, refinement is the reverse of abstraction.

Principle of abstraction: The level of abstraction of the representation of a system can be gradually increased by successively replacing the details with abstract information. By means of abstraction one can produce a view that focuses on selected system characteristics by hiding information about other characteristics.

Principle of refinement: The level of abstraction of the representation of the system is gradually decreased by successively replacing some aspects of the system with more details.

Reengineering Concepts

A new software is created by going downward from the top, highest level of abstraction to the bottom, lowest level. This downward movement is known as **forward engineering**.

Forward engineering follows a sequence of activities: formulating concepts about the system to identifying requirements to designing the system to implementing the design.

On the other hand, the upward movement through the layers of abstractions is called **reverse engineering**.

Reverse engineering of software systems is a process comprising the following steps:

- (i) analyze the software to determine its components and the relationships among the components,
- (ii) represent the system at a higher level of abstraction or in another form.

Decompilation is an example of Reverse Engineering, in which object code is translated into a high-level program.

Reengineering Concepts

The concepts of abstraction and refinement are used to create models of software development as sequences of phases, where the phases map to specific levels of **abstraction** or **refinement**, as shown in Figure 4.1.

The four levels are:

1. Conceptual,
2. Requirements,
3. Design,
4. Implementation.

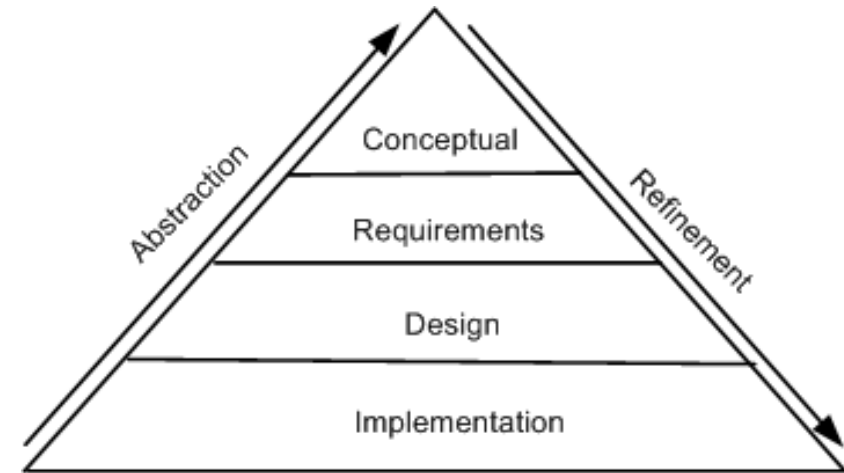


Figure 4.1 levels of abstraction and refinement

The refinement process:

why? ! what? ! what & how? ! how?

The abstraction process:

how? ! what & how? ! what? ! why?

Reengineering Concepts

An optional principle called **alteration** underlies many reengineering methods.

Principle of alteration: The making of some changes to a system representation is known as alteration. Alteration does not involve any change to the degree of abstraction, and it does not involve modification, deletion, and addition of information.

Reengineering principles are represented by means of arrows. Abstraction is represented by an up-arrow, alteration is represented by a horizontal arrow, and refinement by a down-arrow.

The arrows depicting refinement and abstraction are slanted, thereby indicating the increase and decrease, respectively, of system information.

It may be noted that alteration is non-essential for reengineering.

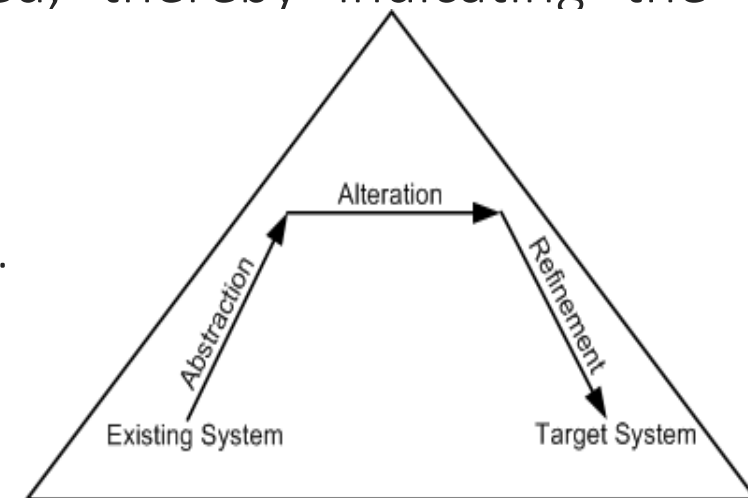


Figure 4.2 Conceptual basis for the reengineering process

General Model For Software Reengineering

The reengineering process accepts as input the existing code of a system and produces the code of the renovated system.

The reengineering process may be as straightforward as translating with a tool the source code from the given language to source code in another language.

For example, a program written in BASIC can be translated into a new program in C.

The reengineering process may be very complex as explained below:

- recreate a design from the existing source code.
- find the requirements of the system being reengineered.
- compare the existing requirements with the new ones.
- remove those requirements that are not needed in the renovated system.
- make a new design of the desired system.
- code the new system.

General Model For Software Reengineering

The model in Figure 4.3 proposed by Eric J. Byrne suggests that reengineering is a sequence of three activities:

- reverse engineering, re-design, and forward engineering
- strongly founded in three principles, namely, abstraction, alteration, and refinement.

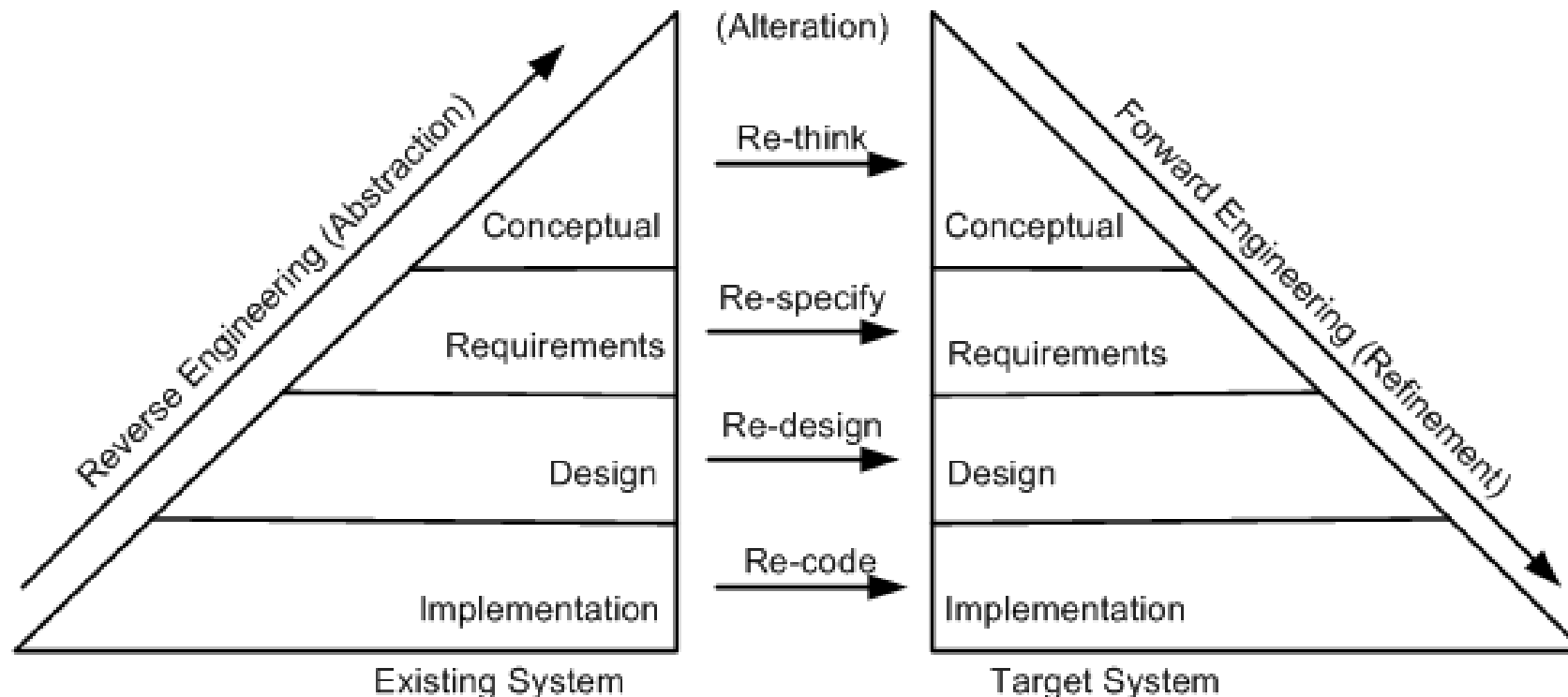


Figure 4.3 General model of software reengineering

General Model For Software Reengineering

A visual metaphor called horseshoe, as depicted in Figure 4.4, was developed by Kazman et al. to describe a **three-step architectural reengineering process**.

Three distinct segments of the horseshoe are the left side, the top part, and the right side. Those three parts denote the three steps of the reengineering process.

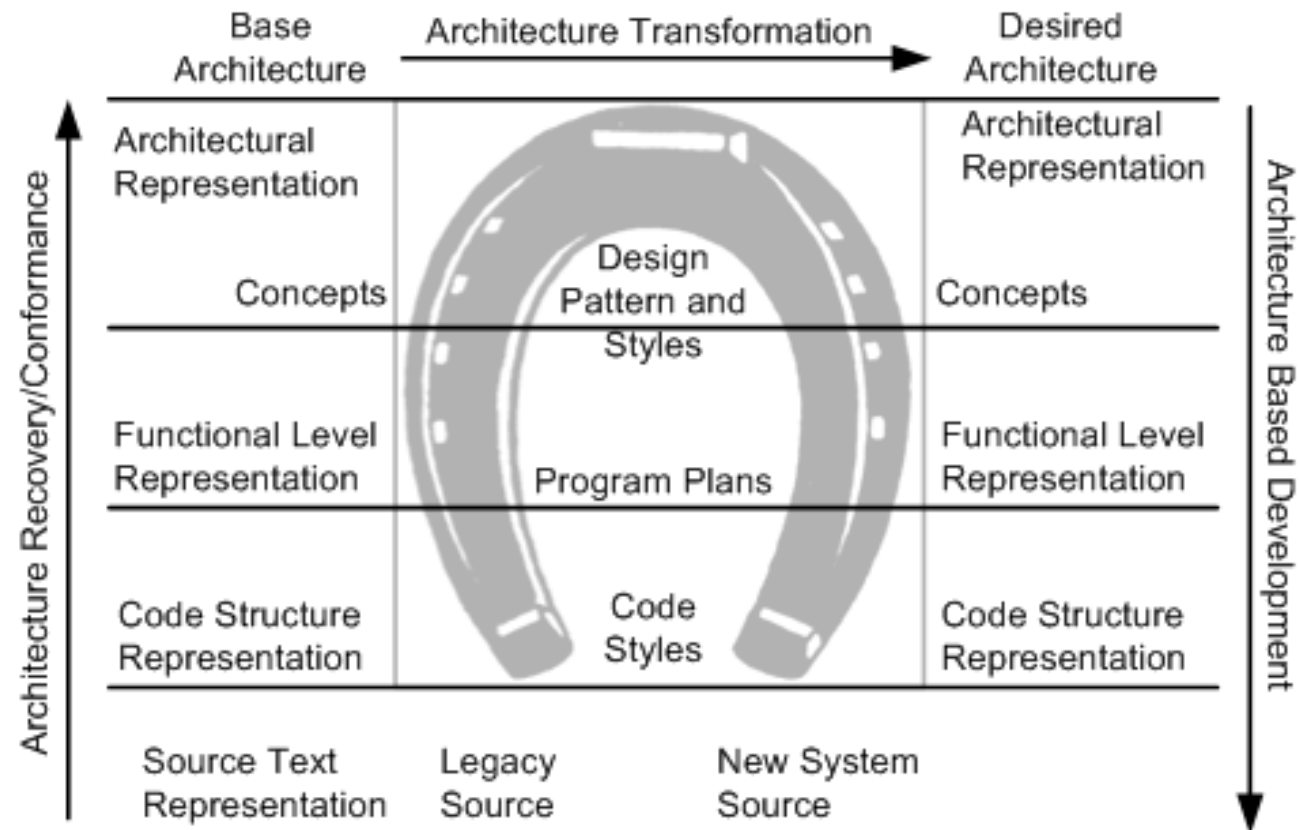


Figure 4.4 Horseshoe model of reengineering

General Model For Software Reengineering

Now, we are in a position to re-visit three definitions of reengineering.

- The definition by Chikofsky and Cross II: Software reengineering is the analysis and alteration of an operational system to represent it in a new form and to obtain a new implementation from the new form. Here, a new form means a representation at a higher level of abstraction.
- The definition by Byrne: Reengineering of a software system is a process for creating a new software from an existing software so that the new system is better than the original system in some ways.
- The definition by Arnold: Reengineering of a software system is an activity that: (i) improves the comprehension of the software system, or (ii) raises the quality levels of the software, namely, performance, reusability, and maintainability.

General Model For Software Reengineering

In summary, it is evident that reengineering entails:

- (i) the creation of a more abstract view of the system by means of some reverse engineering activities,
- (ii) the restructuring of the abstract view, and
- (iii) implementation of the system in a new form by means of forward engineering activities.

This process is formally captured with the following expression:

Reengineering = Reverse engineering + Δ + Forward engineering.

The element “ Δ ” captures alterations made to the original system.

Two major dimensions of alteration are: **change in functionality and change in implementation technique.**

A change in functionality comes from a change in the business rules

Next, concerning a change of implementation technique, an end-user of a system never knows if the system is implemented in an object-oriented language or a procedural language

General Model For Software Reengineering

Another common term used by practitioners of reengineering is **rehosting**.

Rehosting means reengineering of source code without addition or reduction of features in the transformed targeted source code.

Rehosting is most effective when the user is satisfied with the system's functionality, but looks for better qualities of the system.

Examples of better qualities are improved efficiency of execution and reduced maintenance costs.

Types of Change

Based on the type of changes required, system characteristics are divided into groups: **rethink, respecify, redesign**, and **re-code**.

Recode:

Implementation characteristics of the source program are changed by re-coding it. Source-code level changes are performed by means of **rephrasing** and **program translation**.

In the last approach, a program is transformed into a program in a different language.

On the other hand, rephrasing keeps the program in the same language.

Examples of translation scenarios are **compilation, decompilation, and migration**.

Examples of rephrasing scenarios are **normalization, optimization, refactoring, and renovation**.

Redesign:

The design characteristics of the software are altered by re-designing the system. Common changes to the software design include:

- (i) restructuring the architecture;
- (ii) Modifying the data model of the system; and
- (iii) replacing a procedure or an algorithm with a more efficient one.

Types of Change

Respecify:

This means changing the requirement characteristics of the system in two ways:

- (i) change the form of the requirements, and
- (ii) change the scope of the requirements.

Rethink:

Re-thinking a system means manipulating the concepts embodied in an existing system to create a system that operates in a different problem domain.

It involves changing the conceptual characteristics of the system, and it can lead to the system being changed in a fundamental way.

Moving from the development of an ordinary cellular phone to the development of smartphone system is an example of Re-think.

Software Reengineering Strategies

Three strategies that specify the basic steps of reengineering are **rewrite**, **rework**, and **replace**.

Rewrite strategy:

This strategy reflects the principle of alteration. By means of alteration, an **operational system is transformed into a new system**, while preserving the abstraction level of the original system. For example, the Fortran code of a system can be rewritten in the C language.

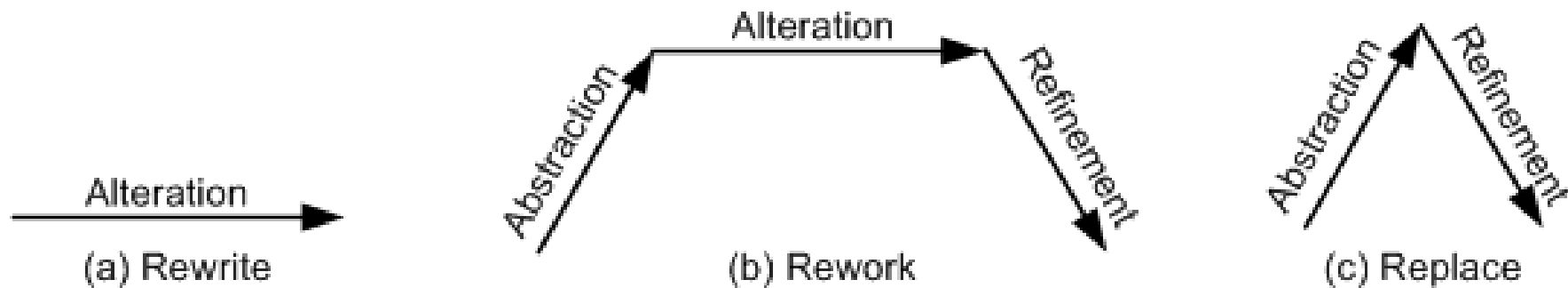


Figure 4.5 Conceptual basis for reengineering strategies

Software Reengineering Strategies

Rework strategy:

The rework strategy applies all the three principles.

Let the goal of a reengineering project is to replace the unstructured control flow constructs, namely GOTOs, with more commonly used structured constructs, say, a “for” loop.

A classical, rework strategy based approach is as follows:

- Application of abstraction: By parsing the code, generate a control-flow graph (CFG) for the given system.
- Application of alteration: Apply a restructuring algorithm to the control-flow graph to produce a structured control-flow graph.
- Application of refinement: Translate the new, structured control-flow graph back into the original programming language.

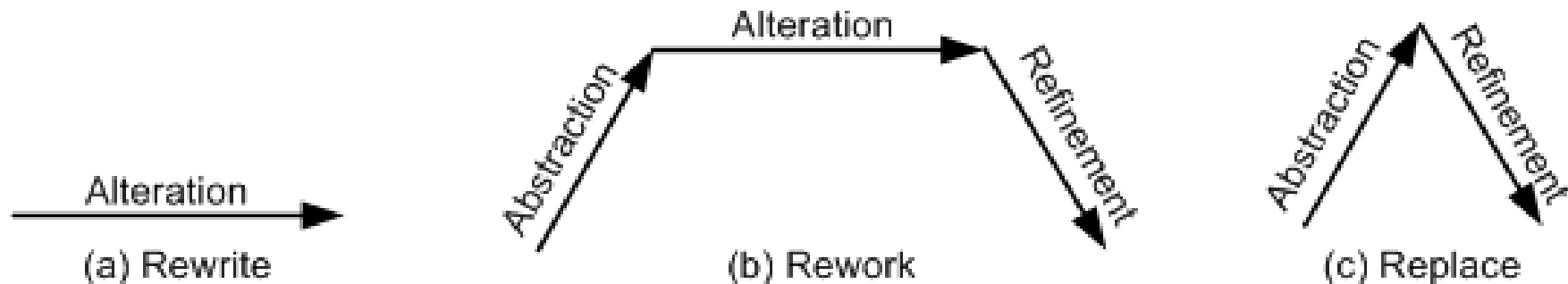


Figure 4.5 Conceptual basis for reengineering strategies

Software Reengineering Strategies

Replace strategy:

The replace strategy applies two principles, namely, abstraction and refinement.

To change a certain characteristic of a system:

- (i) the system is reconstructed at a higher level of abstraction by hiding the details of the characteristic; and
- (ii) a suitable representation for the target system is generated at a lower level of abstraction by applying refinement.

Let us reconsider the GOTO example. By means of abstraction, a program is represented at a higher level without using control flow concepts.

Next, by means of refinement, the system is represented at a lower level of abstraction with a new structured control flow.



Figure 4.5 Conceptual basis for reengineering strategies

Starting Abstraction Level	Type Change	Reengineering Strategy		
		Rewrite	Rework	Replace
Implementation Level	Re-code	Yes	Yes	Yes
	Re-design	Bad	Yes	Yes
	Re-specify	Bad	Yes	Yes
	Re-think	Bad	Yes*	Yes*
Design Level	Re-code	No	No	No
	Re-design	Yes	Yes	Yes
	Re-specify	Bad	Yes	Yes
	Re-think	Bad	Yes*	Yes*
Requirement Level	Re-code	No	No	No
	Re-design	No	No	No
	Re-specify	Yes	Yes	Yes
	Re-think	Bad	Yes*	Yes*
Conceptual Level	Re-code	No	No	No
	Re-design	No	No	No
	Re-specify	No	No	No
	Re-think	Yes	Yes*	Yes*

Table 4.1: Reengineering process variations [5] (©[1992] IEEE).

Yes - One can produce a target system.

Yes* - Same as Yes, but the starting degree of abstraction is lower than the uppermost degree of abstraction within the conceptual abstraction level.

No - One cannot start at abstraction level A , make B type of changes by using strategy C , because the starting abstraction level is higher than the abstraction level required by the particular type of change.

Bad - A target system can be created, but the likelihood of achieving a good result is low.

Reengineering Process

We will discuss five process models for software reengineering.

The five approaches are different in two aspects:

- (i) the extent of reengineering performed, and
- (ii) the rate of substitution of the operational system with the new one.

Reengineering Approaches

Big Bang Approach

The “**Big Bang**” approach replaces the whole system at once.

Once a reengineering effort is initiated, it is continued until all the objectives of the project are achieved and the target system is constructed.

This approach is generally used if reengineering cannot be done in parts.

For example, if there is a need to move to a different system architecture, then all components affected by such a move must be changed at once.

The consequent advantage is that the system is brought into its new environment all at once.

The disadvantage of Big Bang is that the reengineering project becomes a monolithic task, which may not be desirable in all situations.

In addition, the Big Bang approach consumes too much resources at once for large systems, and takes a long stretch of time before the new system is visible.

Reengineering Approaches

Incremental Approach

In this approach a system is reengineered gradually, one step closer to the target system at a time.

For a large system, several new interim versions are produced and released.

Successive interim versions satisfy increasingly more project goals than their preceding versions.

The advantages of this approach are as follows:

- (i) locating errors becomes easier, because one can clearly identify the newly added components, and
- (ii) It becomes easy for the customer to notice progress, because interim versions are released.

The disadvantages of the incremental approach are as follows:

- (i) with multiple interim versions and their careful version controls, the incremental approach takes much longer to complete, and
- (ii) even if there is a need, the entire architecture of the system cannot be changed.

Reengineering Approaches

Partial Approach

In this approach, only a part of the system is reengineered and then it is integrated with the non-engineered portion of the system.

One must decide whether to use a “Big Bang” approach or an “Incremental” approach for the portion to be reengineered.

The following three steps are followed in the partial approach:

- In the first step, the existing system is partitioned into two parts: one part is identified to be reengineered and the remaining part to be not reengineered.
- In the second step, reengineering work is performed using either the “Big Bang” or the “Incremental” approach.
- In the third step, the two parts, namely, the not-to-be-reengineered part and the reengineered part of the system, are integrated to make up the new system.

The partial approach has the advantage of reducing the scope of reengineering that is less time and costs less.

A disadvantage of the partial approach is that modifications are not performed to the interface between the portion modified and the portion not modified.

Reengineering Approaches

Iterative Approach

The reengineering process is applied on the source code of a few procedures at a time, with each reengineering operation lasting for a short time.

This process is repeatedly executed on different components in different stages.

During the execution of the process, ensure that the four types of components can coexist:

1. old components not reengineered,
2. components currently being reengineered,
3. components already reengineered, and
4. new components added to the system.

There are two advantages of the iterative reengineering process:

- (i) it guarantees the continued operation of the system during the execution of the reengineering process, and
- (ii) the maintainers' and the users' familiarities with the system are preserved.

The disadvantage of this approach is the need to keep track of the four types of components during the reengineering process.

In addition, both the old and the newly reengineered components need to be maintained.

Reengineering Approaches

Evolutionary Approach

In the "Evolutionary" approach components of the original system are substituted with re-engineered components.

In this approach, the existing components are grouped by functions and reengineered into new components.

Software engineers focus their reengineering efforts on identifying functional objects irrespective of the locations of those components within the current system.

As a result, the new system is built with functionally cohesive components as needed.

There are two advantages of the "Evolutionary" approach:

- (i) the resulting design is more cohesive, and
- (ii) the scope of individual components is reduced.

A major disadvantage:

- (i) all the functions with much similarities must be first identified throughout the operational system.
- (ii) next, those functions are refined as one unit in the new system.

Source Code Reengineering Reference Model

- The SCORE/RM model was proposed by Colbrook, Smythe and Darlison.
- The framework, depicted in Figure, consists of four kinds of elements:
- The function element is divided into eight layers, namely:

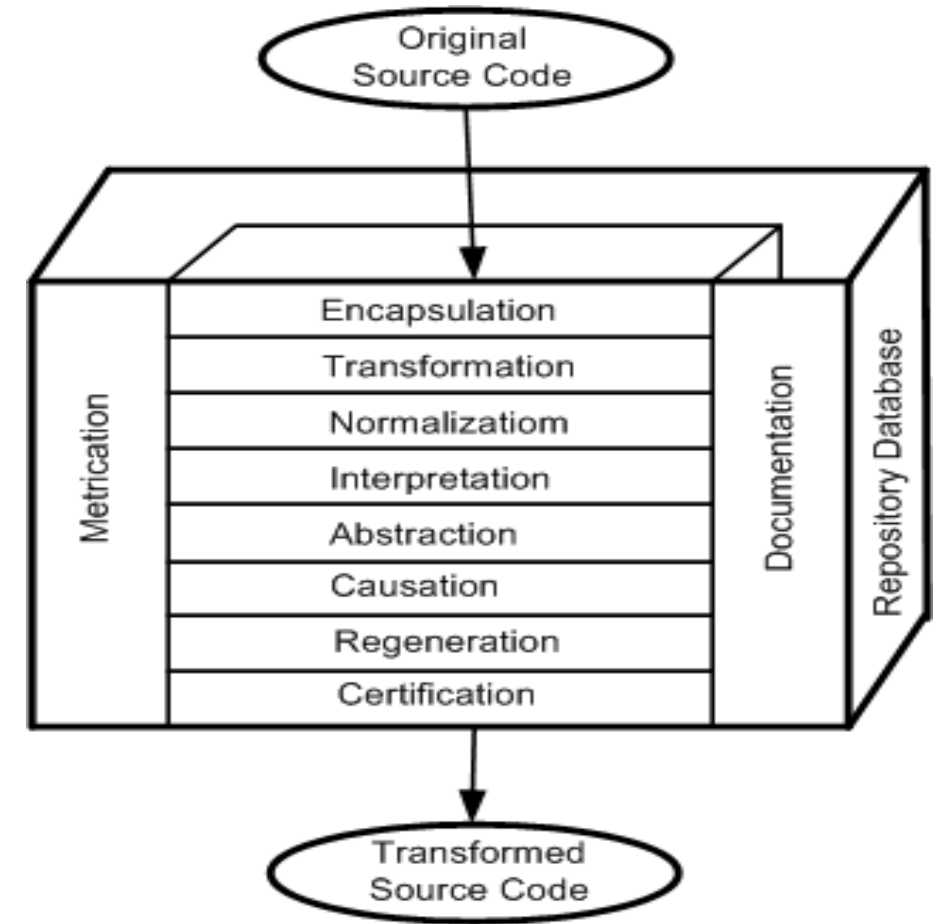


Figure 4.6 Source code reengineering reference model

Source Code Reengineering Reference Model

- The eight layers provide a detailed approach to:
- The first six of the eight layers together constitute a process for reverse engineering, and the final three a process for forward engineering.
- Improvements in the software as a result of reengineering is quantified by means of the metrication element.
- The **metrication element** is described in terms of the relevant software metrics before executing a layer and after executing the same layer.
- The **repository database** is the information store for the entire reengineering process, containing the following kinds of information:

Source Code Reengineering Reference Model

- The interfaces among the elements are shown in Figure.
- For simplicity, any layer is referred to as (N)-layer, while its next lower and next higher layers are referred to as (N - 1)-layer and the (N + 1)-layer, respectively.
- The three types of interfaces are explained as follows:

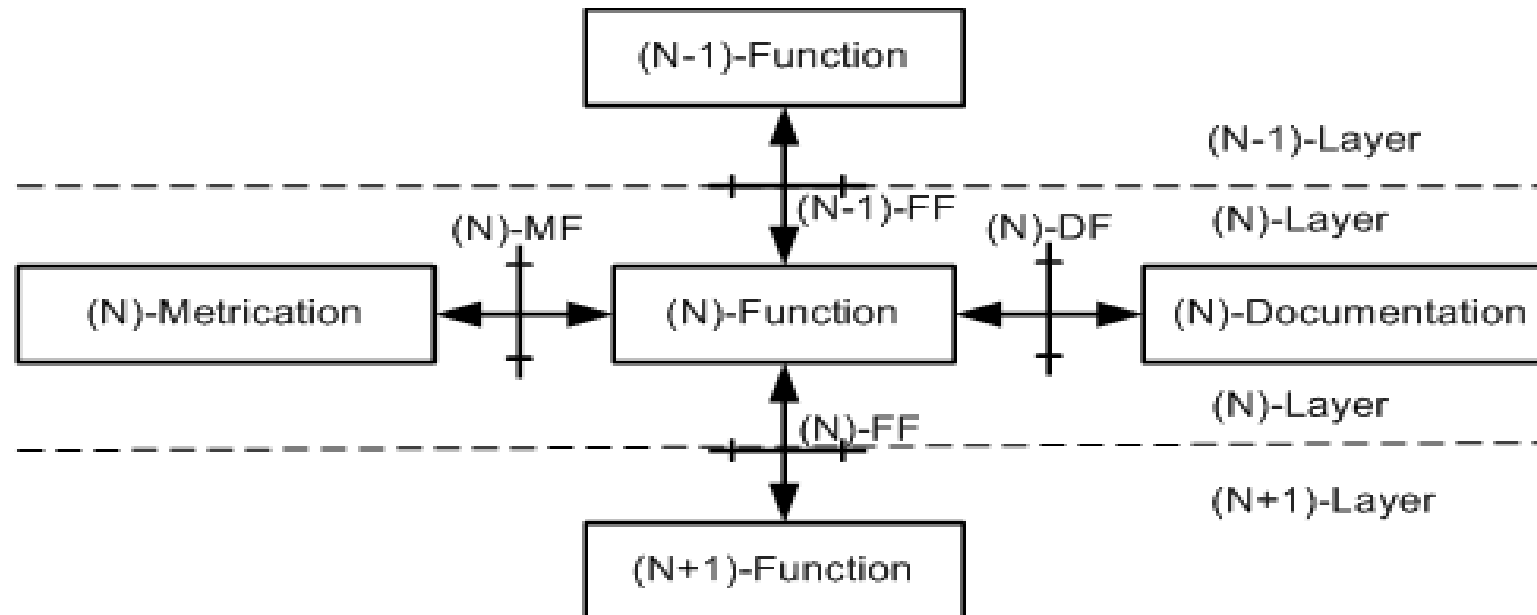


Figure 4.7 The interface nomenclature

Phase Reengineering Model

The phase model of software reengineering was originally proposed by Byrne and Gustafson.

The model comprises five phases: analysis and planning, renovation, target system testing, redocumentation, and acceptance testing and system transition, as depicted in Figure 4.8.

The labels on the arcs denote the possible information that flows from the tail entities of the arcs to the head entities.

- A major process activity is represented by each phase.
- Tasks represent a phase's activities, and tasks can be further decomposed to reveal the detailed methodologies.

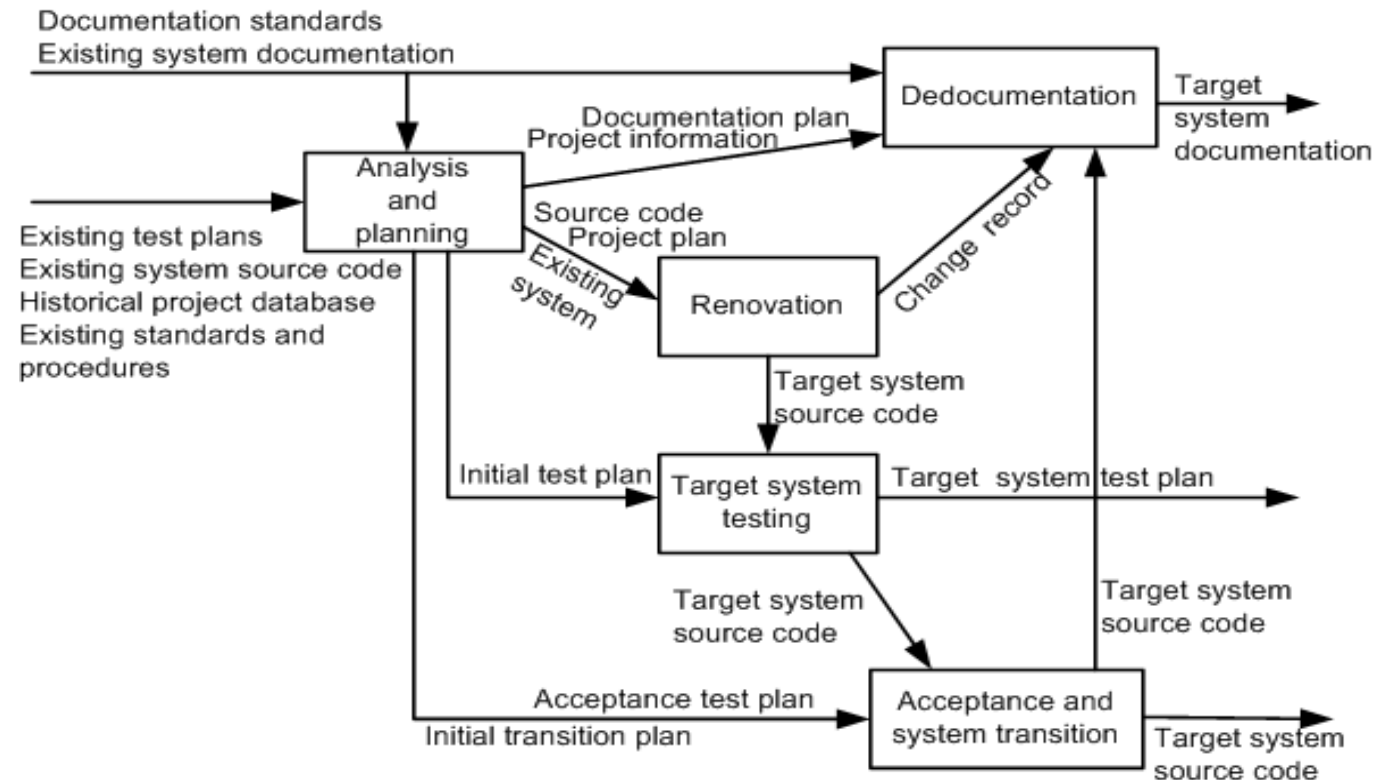


Figure 4.8 Software reengineering process phases

Phase Reengineering Model

Analysis and planning:

Analysis addresses three technical and one economic issue.

The first technical issue concerns the present state of the system to be reengineered and understanding its properties.

The second technical issue concerns the identification of the need for the system to be reengineered.

The third technical issue concerns the specification of the characteristics of the new system to be produced.

The economic issue concerns a cost and benefit analysis of the reengineering project.

Planning includes:

- understanding the scope of the work;
- identifying the required resources;
- identifying the tasks and milestones;
- estimating the required effort; and
- preparing a schedule.

Phase Reengineering Model

TABLE 4.2 Tasks—Analysis and Planning Phase

Task	Description
Implementation motivations and objectives	List the motivations for reengineering the system. List the objectives to be achieved.
Analyze environment	Identify the differences between the existing and the target environments. Differences can influence system changes.
Collect inventory	Form a baseline for knowledge about the operational system by locating all program files, documents, test plans, and history of maintenance.
Analyze implementation	Analyze the source code and record the details of the code.
Define approach	Choose an approach to reengineer the system.
Define project procedures and standards	Procedures outline how to perform reviews and report problems. Standards describe the acceptable formats of the outputs of processes.
Identify resources	Determine what resources are going to be used; ensure that resources are ready to be used.
Identify tools	Determine and obtain tools to be used in the reengineering project.
Data conversion planning	Make a plan to effect changes to databases and files.
Test planning	Identify test objectives and test procedures, and evaluate the existing test plan. Design new tests if there is a need.
Define acceptance criteria	By means of negotiations with the customers, identify acceptance criteria for the target system.
Documentation planning	Evaluate the existing documentation. Develop a plan to redocument the target system.
Plan system transition	Develop an end-of-project plan to put the new system into operation and phase out the old one.
Estimation	Estimate the resource requirements of the project: effort, cost, duration, and staffing.
Define organizational structure	Identify personnel for the project, and develop a project organization.
Scheduling	Develop a schedule, including dependencies, for project phases and tasks.

Phase Reengineering Model

Renovation:

An operational system is modified into the target system in the renovation phase.

Two main aspects of a system are considered in this phase:

(i) representation of the system.

It refers to source code, but it may include the design model and the requirement specification of the existing system.

(ii) representation of external data.

It refers to the database and/or data files used by the system. Often the external data are reengineered, and it is known as data reengineering.

An operational system can be renovated in many ways, depending upon the objectives of the project, the approach followed, and the starting representation of the system.

It may be noted that the starting representation can be source code, design, or requirements.

Table 4.1 discussed earlier recommends several alternatives to renovate a system.

Phase Reengineering Model

Renovation: Example

A project in which the objective is to re-code the system from Fortran to C.

Figure 4.9 shows the three possible replacement strategies.

First, to perform source-to-source translation, program migration is used.

Second, a high-level design is constructed from the operational source code, say, in Fortran, and the resulting design is re-implemented in the target language, C in this case.

Finally, a mix of compilation and decompilation is used to obtain the system implementation in C

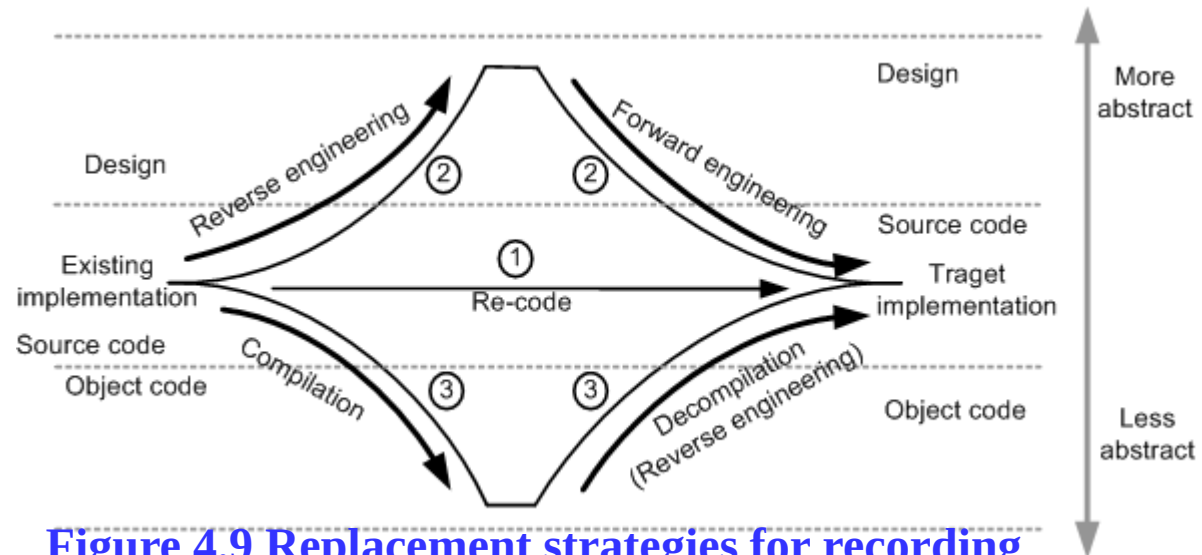


Figure 4.9 Replacement strategies for recording

Phase Reengineering Model

Target system testing:

In this phase for system testing, faults are detected in the target system.

Those faults might have been introduced during reengineering.

Fault detection is performed by applying the target system test plan on the target system.

The same testing strategies, techniques, methods, and tools that are used in software development are used during reengineering.

For example, apply the existing system-level test cases to both the existing and the new system.

Assuming that the two systems have identical requirements, the test results from both the scenarios must be the same.

Phase Reengineering Model

Redocumentation:

In the redocumentation phase, documentations are rewritten, updated, and/or replaced to reflect the target system.

Documents are revised according to the redocumentation plan.

The two major tasks within this phase are:

- (i) analyze new source code, and
- (ii) create documentation.

Documents requiring revision are:

- requirement specification.
- design documentation.
- a report justifying the design decisions, assumptions made in the implementation.
- configuration.
- user and reference manuals.
- on-line help.
- document describing the differences between the existing and the target system.

Code Reverse Engineering

Reverse engineering was first applied in electrical engineering to produce schematics from an electrical circuit.

It was defined as the process of developing a set of specifications for a complex hardware system by an orderly examination of specimens of that system.

In the context of software engineering, Chikofsky and Cross II defined reverse engineering as a process to:

- (i) identify the components of an operational software.
- (ii) identify the relationships among those components.
- (iii) represent the system at a higher level of abstraction or in another form.

Reverse engineering is performed to achieve two key objectives:

redocumentation of artifacts

It aims at revising the current description of components or generating alternative views at the same abstraction level. Examples of redocumentation are pretty printing and drawing CFGs.

design recovery

It creates design abstractions from code, expert knowledge, and existing documentation.